# LLMs and Transformers

Sri Pranav Kunda

March 2025

# Contents

1. Introducing Transformers
2. Training LLMs
3. Fine-Tuning LLMs
4. Universal Approximation Theorems for Transformers

# Introducing Transformers

# What is an LLM

- A language model $f$ assigns probabilities to words or sequences of words.
- Given a sequence (vector) of tokens $x_{[T]} = x_1, \ldots x_T$, $f(\mathbf{x}_{[T]})$ is a probability over the list of known tokens that can be used to determine the next token $x_{T+1}$.
- Language models can attain high levels of expressive power when trained on large sets of text.

# Understanding Transformers: Queries, Keys, and Values

- Pre-trained language models have mostly moved to the Transformer architecture because of their success in capturing long-range contextual relationships in text.
- The key architectural components of a Transformer are the encoder, decoder, and self-attention blocks

# Understanding Transformers: Queries, Keys, and Values

Provided a set of keys and values $\mathcal{D} = \{(k_1, v_1), \ldots, (k_n, v_n)\}$ and a query $q$, we write

$$\text{Attention}(q, \mathcal{D}) = \sum \alpha(q, k_i) v_i,$$

where $\alpha$ is some kernel. The kernel is typically normalized so that $\sum_i \alpha(q, k_j) = 1$. This operation is called attention pooling.

# Understanding Transformers: Queries, Keys, and Values

The attention mechanism then depends on the choice of attention pooling mechanism. Some common choices are:

- Scaled Dot Product Attention:

$$\alpha(q, k_i) = \text{softmax}\left(\frac{q^\top k_i}{\sqrt{d}}\right)$$

with $q, k_i \in \mathbb{R}^d$.

# Understanding Transformers: Queries, Keys, and Values

The attention mechanism then depends on the choice of attention pooling mechanism. Some common choices are:

- Additive Attention (for when $q$ and $k_i$ don't match in dimensionality):

$$\alpha(q, k_i) = \text{softmax}(w_v^\top \tanh(W_q q + W_k k_i)),$$

where $W_q$, $W_v$, and $w_v$ are learnable parameters.

# Understanding Transformers: The Encoder-Decoder Architecture

1. The encoder-decoder architecture is commonly applied to sequence-to-sequence problems (e.g. machine translation, text generation).
2. Given an RNN and its output states $h_{[T]}$, the encoder generates a context variable $c = g(h_{[T]})$
3. The decoder takes the current model state, the context variable $c$, and the previously generated token to output a probability vector over the list of known tokens.

# Understanding Transformers: Decoders with Attention

Instead of the decoder only generating a singular context variable from the state of the model, the attention model can be used to generate a context variable $c_t$ for every recurrent step $t$.

In particular, if $H = \{h_1, \ldots, h_T\}$ is the set of states,

$$c_{t'} = \text{Attention}(s_{t'-1}, H \times H) = \sum_{t=1}^{T} \alpha(s_{t'-1}m, h_t)h_t.$$

# Understanding Transformers: Attention Pooling in Minibatches

Note that we can often use minibatches to rewrite attention pooling as a single operation. For instance, the scaled dot product attention of queries $Q$, $V$, and $K$ can be written as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V.$$

# Understanding Transformers: Multi-head Attention

The idea behind multi-head attention is that we can use different representation subspaces of our input sequence to capture different information from the input sequence. To do this, $W_i^{(q)}, W_i^{(k)}, W_i^{(v)}$ be learnable parameters. Then given a query $q$, key $k$, and $v$, we can compute the output of each attention head $b_i$ as

$$b_i = \text{Attention}(W_i^{(q)}q, W_i^{(k)}k, W_i^{(v)}v).$$

The final multi-head output is then given by another learnable parameter $W_o$ as

$$W_o \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix}$$

# Understanding Transformers: Self-Attention and Positional Encoding

The self-attention of a sequence $x_{[T]} = x_1, \dots x_T$ is $y_{[T]} = y_1, \dots y_T$ with

$$y_i = \text{Attention}(x_i, \{(x_1, x_1), \dots, (x_T, x_T)\}).$$

This operation does not preserve the order of the original sequence $x_{[T]}$, so positional encodings are typically added (either through a computation or through learning) as data to every token.

# The Transformer Architecture

A transformer contains stacks of sublayers, each of which is either a positionwise feed-forward network or multi-head self-attention pooling block.

# The Transformer Architecture

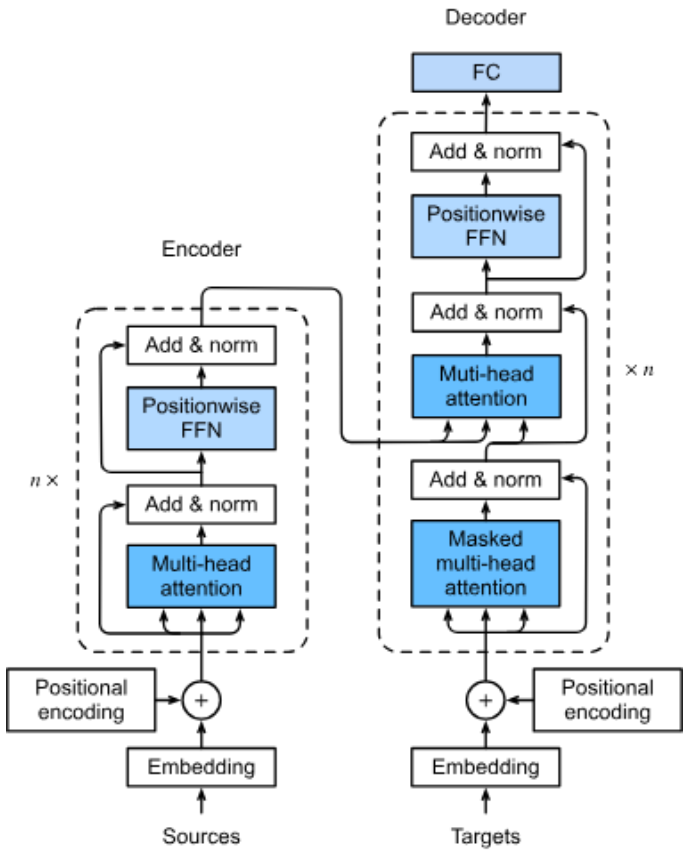



Figure: Transformer Architecture (Source: Dive into Deep Learning)

# The Decoder-Only Transformer Architecture

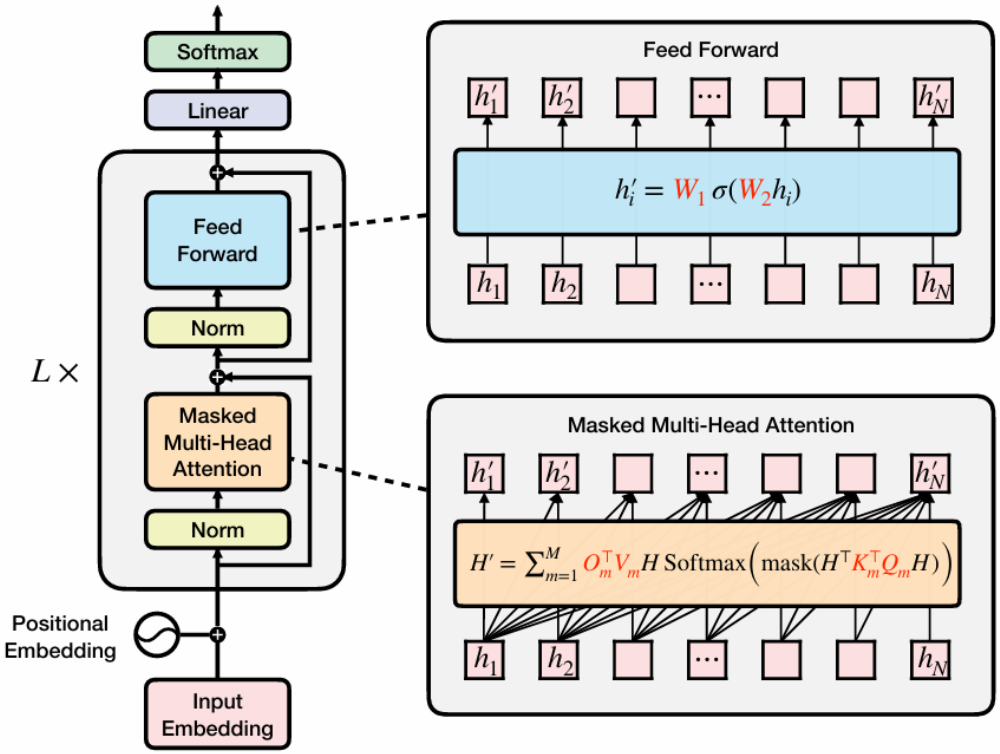



Figure: Decoder-Only Transformer Architecture (Source: An Overview of Large Language Models for Statisticians)

# Training LLMs

# The Pretrain-Finetune Paradigm

- The modern method to training a large language model is to do so in two stages: pre-training and fine-tuning.
- First, a model containing a fixed architecture is trained through on some large dataset of text.
- Then, the model is "fine-tuned" by adding learnable parameters and training these parameters on specific objective functions.

# The Pre-Training Objective

The pre-training objective is typically for a language model parametrized by $\theta$ with context $c_t$ is to minimize the conditional probability $\mathbb{P}_\theta(x_t \mid c_t)$.

The commonly used loss function for achieving this is $\mathcal{L}_{SLM} = -\sum_{t=1}^{T} \log \mathbb{P}_\theta(x_t \mid c_t)$.

# Choosing Pre-Training Data

- The language data on which an LLM is trained makes a large impact on its performance
- It has been empirically shown that the inclusion of code data in pre-training helps not only in tasks involving programming but also in non-programming tasks.
- Heuristic or optimization methods are often used to select the tokens and samples used for pre-training.

# Fine-Tuning LLMs

# LLM Prompting

For particularly large language models, traditional supervised learning used for fine-tuning is replaced with prompt engineering.

In supervised learning, labeled data $\mathcal{D} = \{(x_i, y_i)\}$ is used to tune the model parameter $\theta$ so that $\mathbb{P}_\theta(y \mid x)$ is optimal for an expected output sequence $y$ and input sequence $x$.

Prompt engineering instead involves finding an input $x'$ such that $\mathbb{P}_\theta(y \mid x)$ is optimal without modifying the model parameter $\theta$.

# In-Context Learning

In-context learning allows for providing "demonstrations" of a task to a language model, with the expectation that the language model will perform the task on an input which is not included in the demonstrations.

# In-Context Learning

In particular, provided demonstrations with input-output pairs $\{(x_i, y_i)\}$ formatted with a common template $t$, an input $x$ formatted using $t$ and appended to the list of demonstrations to form a new input.

This is then equivalent to the language model estimating the probability $\mathbb{P}_\theta(y \mid x_1, y_1, \ldots, x_n, y_n, x)$

# Parameter-Efficient Fine Tuning

There are multiple strategies of fine-tuning which involve modifying only a subset of the model parameters. Some commonly used methods are:

- **Additive Fine Tuning:** A small number of learnable parameters are added (whose positions are chosen optimally) to the model.
- **Selective Fine Tuning:** A binary mask is added to the model's parameters which indicates which of the parameters should be learned during fine-tuning.

# Universal Approximation Theorems for Transformers

# Permutation Equivariance of Transformers

A function $f : \mathbb{R}^{m \times n} \to \mathbb{R}^{m \times n}$ is permutation equivariant if $f(XP) = f(X)P$ for any permutation matrix $P$.

**Theorem (Permutation Equivariance of Transformers)**

*The map $T$ given by a transformer block is permutation equivariant.*

# The Class $\mathcal{F}_{PE}$

**Definition (The Class $\mathcal{F}_{PE}$)**

Let $\mathcal{F}_{PE}$ be the class of all continuous, compactly supported, and permutation equivariant functions which map $\mathbb{R}^{m\times n} \to \mathbb{R}^{m\times n}$. A function is said to be compactly supported if the closure of $\{x \mid f(x) \neq 0\}$ is compact.

Also define

$$d_p(f,g) = \left( \int \|f(X) - g(X)\|_p^p \, dX \right)^{1/p}.$$

# Transformers are Universal Approximators

**Theorem (First Universal Approximation Theorem)**

*Let $1 \leq p < \infty$ and $\epsilon > 0$. Then there exists a transformer $T$ for any $f \in \mathcal{F}_{PE}$ such that $d_p(T, f) < \epsilon$.*

# The Class $\mathcal{F}_{CD}$

**Definition (The Class $\mathcal{F}_{CD}$)**

Let $\mathcal{F}_{CD}$ be the class of all continuous which map a compact subset of $\mathbb{R}^{m \times n}$ to $\mathbb{R}^{m \times n}$.

# Transformers are Universal Approximators

**Theorem (Second Universal Approximation Theorem)**

*Let $1 \leq p < \infty$ and $\epsilon > 0$. Then there exists a transformer $T$ with positional encodings for any $f \in \mathcal{F}_{CE}$ such that $d_p(T, f) < \epsilon$.*